Reviewer Pack — Minimal Rank of Noncommutative Differential Forms vs AC0 Par…

Entry 0486e910823a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 03:29:01 UTC

Minimal Rank of Noncommutative Differential Forms vs AC0 Parity Circuit Threshold

Entry ID: `0486e910823a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 03:29:01 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Differential Geometry **Field B** (complexity object): Complexity Theory: AC0 Parity Circuit Complexity

Statement:

> For any AC0 parity circuit C with size n, the minimal rank of its associated noncommutative differential form is at least c·log(n), where c is a constant.

Rationale (proposer's reasoning):

> Noncommutative differential geometry offers a rich framework for studying geometric structures on algebraic structures. The minimal rank of a noncommutative differential form could provide a geometric invariant that captures the complexity of AC0 parity circuits, potentially replacing the random restriction argument with a more structured one.

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `133b2db7bafbc203`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The mean minimal rank of noncommutative differential forms associated with AC0 parity circuits is greater than or equal to c·log(n), where c is a constant.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_parity_circuit(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_noncommutative_differential_form(circuit):
        # Placeholder function to simulate computation
        return sum(circuit) % 2

    n = random.randint(5, 40)
    circuit = generate_ac0_parity_circuit(n)
    rank = compute_noncommutative_differential_form(circuit)

    c = 1
    if rank < c * math.log(n):
        return {
            "metric_name": "Minimal Rank of Noncommutative Differential Form",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"c={c} too small"
        }
    else:
        return {
            "metric_name": "Minimal Rank of Noncommutative Differential Form",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": True,
            "counterexample": ""
        }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
```

```python
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"c too small\" first_failing_seed={first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13232
2	preregistration	ollama_remote	glm4:latest	0	0	8813
3	novelty	ollama_remote	glm4:latest	0	0	9247
4	novelty	ollama_remote	glm4:latest	0	0	10484
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12830
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29116
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8016
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8477
9	judge	ollama_remote	glm4:latest	0	0	52543

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 152759 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0486e910823a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0486e910823a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0486e910823a.tar.gz` (if generated)